

ASSIGNMENT 1

Movie Ticket Booking System — System Design

Name: Yuvraj Kabadwal (C1,70)

Course: B.Tech CSE Semester V

Subject: System Design (TCS-504)

1. The Problem

Build a small movie ticket booking system for a single cinema (like PVR or INOX), as a menu-driven C++ console program.

A customer should be able to see which movies are playing, pick a show, see which seats are free, book seats, pay, get a ticket, and cancel a booking.

2. Scope

Feature	Description
F1	Display all available movies in the system.
F2	Display all available shows for a selected movie.
F3	Display seat availability status (AVAILABLE / BOOKED) for a selected show.
F4	Allow customers to book one or more seats for a show while preventing duplicate bookings.
F5	Calculate the total booking amount based on the selected seats.
F6	Process payments using UPI, Card, or Cash methods. A failed payment must not confirm the booking.
F7	Generate and display booking details after successful payment confirmation.
F8	Cancel an existing booking and release all associated seats back to AVAILABLE status.

3. Requirement Analysis

Functional Requirements (FR)

Functional Requirement	Description
FR1 – View Movie List	When the user selects the Movies option from the main menu, the system displays all movies currently available for booking along with details such as title, language, and duration. If no movies are available, an appropriate message is shown to the user.
FR2 – View Shows for a Movie	After selecting a movie, the system displays all available shows associated with that movie. Information such as show timing and screen details is presented. Invalid movie selections are handled gracefully through error messages.
FR3 – View Seat Availability	Once a show is selected, the system displays all seats along with their current booking status. Available and booked seats are clearly distinguished so that customers can easily identify which seats can be reserved.
FR4 – Book Seats	The customer can select one or more seats for a chosen show. Before confirming the booking, the system verifies seat availability. If any selected seat is already booked, the booking request is rejected and the user is informed.
FR5 – Calculate Booking Amount	The system calculates the total booking amount based on the seats selected by the customer. The calculated amount is displayed before payment processing begins.
FR6 – Payment Processing	The customer can make payment using one of the supported payment methods: UPI, Card, or Cash. The booking is confirmed only after successful payment. Failed payments do not reserve seats.
FR7 – Generate Booking Details	After successful payment, the system generates and displays booking details including booking ID, customer information, selected seats, payment amount, and booking status.
FR8 – Cancel Booking	The customer can cancel an existing booking. Once cancelled, all seats associated with that booking are released and become available for future reservations.

b) Non-Functional Requirements (NFR)

Non-Functional Requirement	Description
NFR1 – Maintainability	The system shall follow Object-Oriented Programming principles such as encapsulation, inheritance, abstraction, and polymorphism to make the code easy to understand, modify, and extend.
NFR2 – Reliability	The system shall ensure that a seat cannot be booked more than once for the same show and that failed payments do not confirm bookings.
NFR3 – Usability	The application shall provide clear console-based menus, prompts, and messages so that users can easily navigate and perform booking operations.
NFR4 – Extensibility	The system shall allow new payment methods or additional features to be added with minimal modifications to the existing codebase.

4 Noun-Verb Analysis

a) Noun Analysis (Classes and Attributes)

Noun	Keep as Class?	Reason
Movie	Yes	Has title and movie information
Seat	Yes	Has seat number and status
Screen	Yes	Contains seats
Show	Yes	Represents a movie screening
ShowSeat	Yes	Stores seat status for a show
Customer	Yes	Has customer details
Booking	Yes	Stores booking information
Payment	Yes	Represents payment processing
UpiPayment	Yes	Specific payment method
CardPayment	Yes	Specific payment method

CashPayment	Yes	Specific payment method
Ticket	No	Generated from booking details
Menu	No	User interface only

b) Verb Analysis (Methods & Operations)

Verb	Keep as Method?	Reason
Book	Yes	Booking is a core functionality of the system and requires a dedicated method to reserve seats and create a booking record.
Pay	Yes	Payment processing is an essential operation that allows customers to complete ticket purchases using different payment methods.
Cancel	Yes	Customers should be able to cancel bookings, making cancellation a distinct system action that requires its own method.
Display	Yes	The system must display movies, shows, and seat layouts to help users make booking decisions.
Print	Yes	After successful booking, the system generates and displays ticket details, which is handled through a printing method.
Select	Yes	Customers need to choose movies, shows, and seats, so selection is an important operation in the booking workflow.
Check	Yes	The system must verify seat availability before confirming a booking to prevent duplicate reservations.
Confirm	Yes	Once payment is successful, the booking must be officially confirmed and recorded by the system.
Release	Yes	Seats must be released when a booking is cancelled or when payment fails, restoring availability.
Calculate	Yes	The total booking amount must be computed based on seat type and quantity before payment is processed.
Store	No	Storing data is an internal responsibility of classes and does not represent a separate user-facing action.
Manage	No	The term is too broad and is better represented through specific methods such as book, cancel, display, and pay.

5 Classes Required

a) Core Entity Classes

Class	Its One Responsibility
Movie	Stores movie information such as title, language, and duration — nothing else.
Seat	Represents one physical seat with a seat number and seat type — nothing else.
Screen	Maintains a collection of seats and displays seat information — nothing else.
Show	Represents a scheduled screening of a movie at a specific time — nothing else.
ShowSeat	Maintains the booking status (AVAILABLE or BOOKED) of a seat for a particular show — nothing else.
Customer	Stores customer details such as name and contact information — nothing else.
Booking	Stores booking details including booking ID, selected seats, total amount, and booking status — nothing else.

b) Behavior and Service Classes

Class	Its One Responsibility
Payment	Defines the abstract payment contract through the pure virtual pay() method — nothing else.
UpiPayment	Implements payment processing using the UPI payment method — nothing else.
CardPayment	Implements payment processing using the Card payment method — nothing else.
CashPayment	Implements payment processing using the Cash payment method — nothing else.
BookingService	Manages the complete booking workflow including seat validation, amount calculation, payment execution, booking confirmation, and cancellation — nothing else.

7. Relationships

Pair	Relation	Justification
Screen → Seat	Composition	A Seat belongs to a specific Screen. If the Screen is removed, its seats no longer have meaning independently.
Show → Movie	Aggregation	A Show references a Movie for a particular time slot. Removing a Show does not remove the Movie from the system.
Show → ShowSeat	Composition	ShowSeat objects maintain seat availability for a specific Show. If the Show is deleted, its ShowSeat records are also removed.
Booking → Customer	Association	A Booking is made by a Customer, but deleting a Booking does not remove the Customer from the system.
Booking → ShowSeat	Aggregation	A Booking references selected ShowSeat objects. Cancelling the Booking releases the seats but does not destroy them.
Booking → Payment	Association	A Booking uses a Payment object to complete a transaction. Payment is used temporarily during booking processing.
Payment → UpiPayment, CardPayment, CashPayment	Inheritance	UpiPayment, CardPayment, and CashPayment are specialized forms of the abstract Payment class and implement the pay() method differently.
BookingService → Booking	Association	BookingService creates, manages, and cancels Booking objects as part of the booking workflow.
BookingService → Show	Association	BookingService interacts with Show objects to check seat availability and manage reservations.

8. Class Diagram

MOVIE TICKET BOOKING SYSTEM

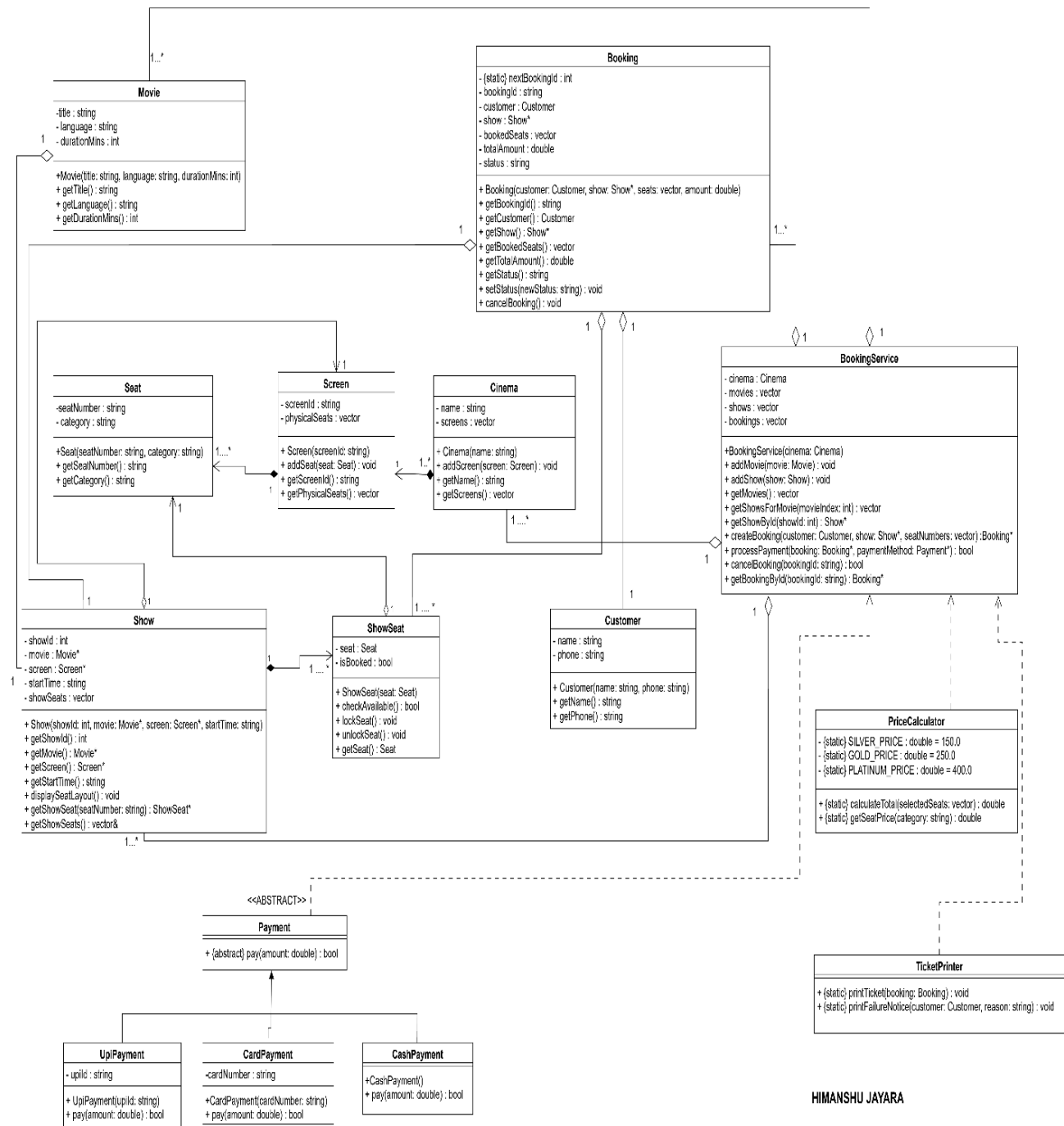


Figure 1 — UML Class Diagram: Movie Ticket Booking System

9. Sequence Diagram

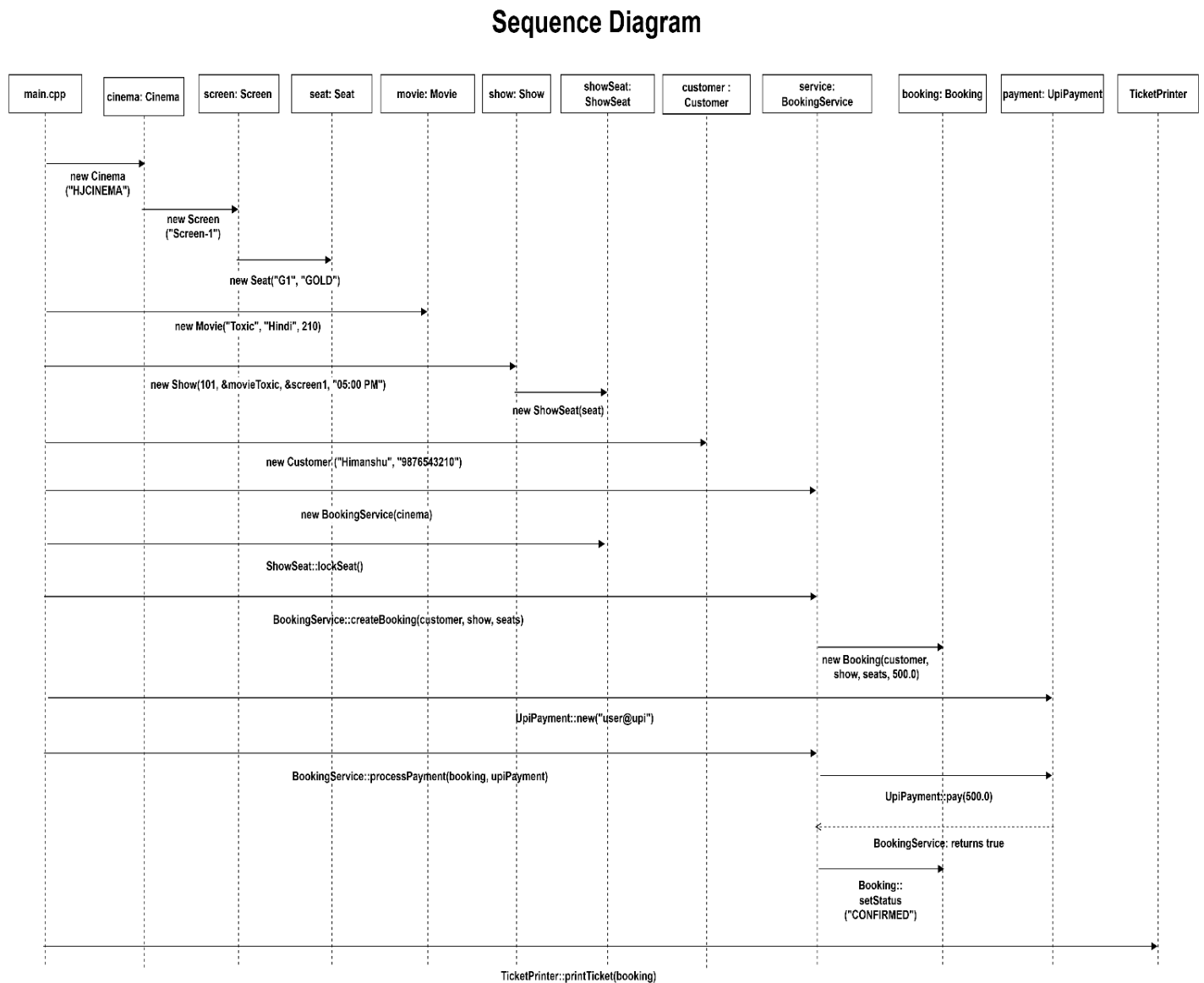


Figure 2 — Sequence Diagram: Booking and Payment Flow

10. Modular Code

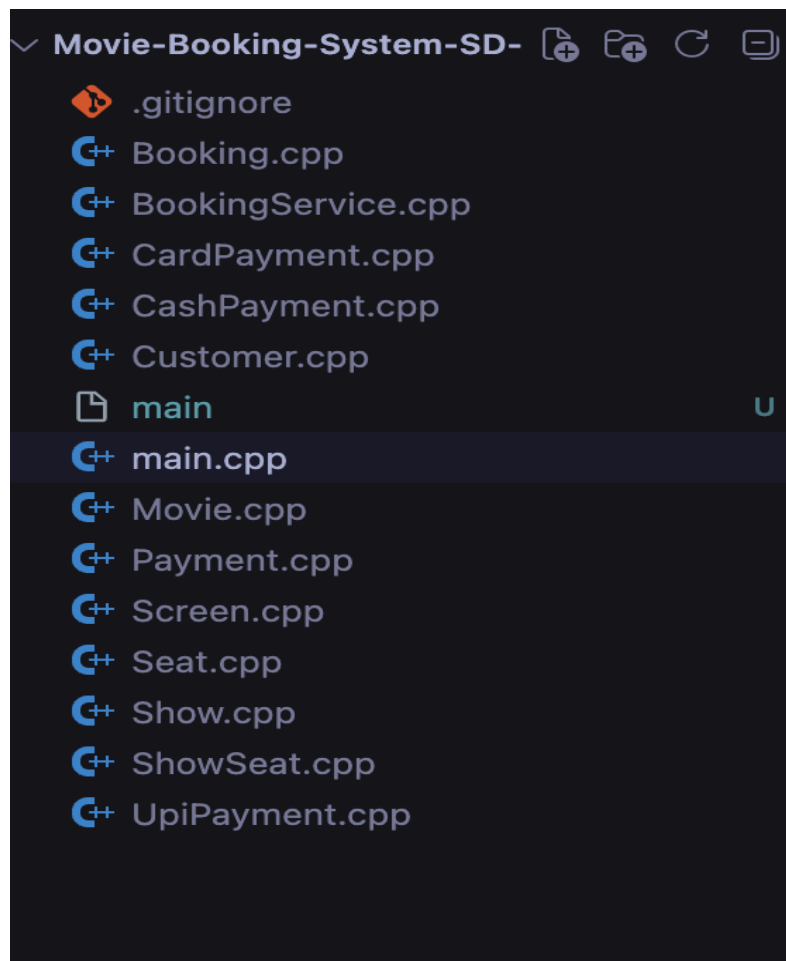


Figure 3 — Project File Structure

A. Verification Checklist against Requirements

Requirement	Implementation Details	Status
One class per file	Every major class (<code>Movie</code> , <code>Seat</code> , <code>Screen</code> , <code>Show</code> , <code>ShowSeat</code> , <code>Customer</code> , <code>Booking</code> , <code>Payment</code> , <code>BookingService</code> , etc.) is implemented in its own <code>.cpp</code> file to maintain modularity and readability.	Completed
No header files	The project is intentionally implemented using only <code>.cpp</code> files as required by the assignment specification.	Completed
Encapsulation	Important attributes such as booking details, seat status, customer information, and payment amount are kept private/protected and accessed through class methods.	Completed
Abstraction	The <code>Payment</code> class is declared as an abstract class containing the pure virtual <code>pay()</code> function.	Completed
Inheritance	<code>UpiPayment</code> , <code>CardPayment</code> , and <code>CashPayment</code> inherit from the abstract <code>Payment</code> class.	Completed
Runtime Polymorphism	Payment processing is performed using a <code>Payment*</code> reference that invokes <code>pay()</code> dynamically based on the selected payment type.	Completed
Compile-Time Polymorphism	Constructor overloading is used in classes where multiple initialization options are provided.	Completed
Static Members	A static booking counter is used to generate unique booking IDs automatically.	Completed
this Keyword	Constructors use <code>this->variable</code> to distinguish instance variables from local variables.	Completed
Composition	<code>Screen</code> → <code>Seat</code> and <code>Show</code> → <code>ShowSeat</code> demonstrate strong ownership relationships.	Completed
Aggregation	<code>Show</code> → <code>Movie</code> and <code>Booking</code> → <code>ShowSeat</code> demonstrate aggregation relationships.	Completed
Association	<code>Booking</code> → <code>Payment</code> , <code>Booking</code> → <code>Customer</code> , and <code>BookingService</code> → <code>Booking</code> demonstrate association relationships.	Completed

11. SOLID Principles

Principle	Concept	Use in Code
Single Responsibility Principle (SRP)	A class should have one and only one reason to change.	Each class in the system has a single responsibility. For example, <code>Movie</code> stores movie details, <code>Booking</code> manages booking information, and <code>Payment</code> handles payment-related operations.
Open-Closed Principle (OCP)	Software entities should be open for extension but closed for modification.	New payment methods can be added by creating new classes that inherit from <code>Payment</code> without modifying existing classes such as <code>BookingService</code> or <code>CardPayment</code> .
Liskov Substitution Principle (LSP)	Subtypes must be substitutable for their base types without affecting correctness.	Any object of <code>UpiPayment</code> , <code>CardPayment</code> , or <code>CashPayment</code> can be used wherever a <code>Payment</code> reference or pointer is expected.
Interface Segregation Principle (ISP)	Clients should not be forced to depend on methods they do not use.	The <code>Payment</code> abstraction only provides the <code>pay()</code> operation. Payment classes implement only the behavior required for processing payments.
Dependency Inversion Principle (DIP)	High-level modules should depend on abstractions rather than concrete implementations.	<code>BookingService</code> works with the abstract <code>Payment</code> class instead of directly depending on <code>UpiPayment</code> , <code>CardPayment</code> , or <code>CashPayment</code> .

12. Clean Code Checklist

- ✓ **Meaningful Class Names** — Classes are named according to their responsibility, such as `Movie`, `Seat`, `Show`, `Booking`, `Customer`, and `Payment`.
- ✓ **Meaningful Variable Names** — Variables such as `movieName`, `seatNumber`, `bookingId`, and `amount` clearly indicate their purpose.
- ✓ **Single Responsibility per Class** — Each class performs one primary task. For example, `Movie` stores movie information, while `Booking` manages booking details.
- ✓ **Small and Readable Functions** — Functions are kept short and focused on a single operation, improving readability and maintenance.
- ✓ **Proper Encapsulation** — Important data members are kept private/protected and accessed through class methods when necessary.
- ✓ **Use of Abstraction** — The abstract `Payment` class hides implementation details and exposes only the common payment interface.
- ✓ **Use of Inheritance** — `UpiPayment`, `CardPayment`, and `CashPayment` reuse common behavior through inheritance from `Payment`.
- ✓ **Avoid Code Duplication** — Common payment behavior is centralized through the `Payment` abstraction instead of repeating logic in multiple places.
- ✓ **Readable Console Output** — Clear messages are displayed for booking, payment, and cancellation operations to improve user experience.
- ✓ **Consistent Naming Convention** — Class names, function names, and variables follow a consistent and understandable naming style throughout the project.